```
a = [1,2,3]
b = [1,2,3]

print(a == b)
print(a is b)
```

Output

```
True
False
```

—> ‘a == b’ checks value equality and returns True.
—> ‘is’ checks memory identity and returnsFalse.

==	is
Compares values	Compares object identity
Return ‘True’ when values are equal	Returns ’True’ when both variables reference the same object
Common use: comparing data	Common use: comparing with None or shared reference

——

Day 2

if-elif-else statements:

They are conditional statements used to perform decision-making in a program. They allow different blocks of code to execute based on whether a condition is True or False.

1. “***if***” statement: it is used to execute a block of code only if the given condition is True. If condition is False, code inside the block is skipped.

```
i = 10

if i > 15:
    print("10 is less than 15")

print("I am Not in if")
```

Output

```
I am Not in if
```

2. "***if-else***" statement: it is used to execute a block of code when condition is True, and another block of code when condition is False.

```
i = 20

if i > 0:
    print("i is positive")
else:
    print("i is 0 or Negative")
```

Output

```
i is positive
```

3. "***if-elif-else***" statement: it is used to check multiple conditions one by one. When a condition is True, its corresponding block of code is executed. If none of the conditions are True, the 'else' block runs.

```
i = 25

if i == 10:
    print("i is 10")

elif i == 15:
    print("i is 15")

elif i == 20:
    print("i is 20")

else:
    print("i is not present")
```

Output

```
i is not present
```

“for” loops:

It is used to iterate over a sequence of elements such as list, tuple, string, range. It executes a block of code once for each element in the sequence.

```
n = 4
for i in range(0, n):
    print(i)
```

Output

```
0
1
2
3
```

“while” loops:

It repeatedly executes a block of code as long as the given condition remains True. When the condition becomes false, the line immediately after the loop is executed.

```
cnt = 0
while (cnt < 3):
    cnt = cnt + 1
    print("Hello Geek")
```

Output

```
Hello Geek
Hello Geek
Hello Geek
```

Nested loops:

It is a loop inside another loop. The inner loop executes completely for every iteration of the outer loop.

```
for i in range(1, 5):
    for j in range(i):
        print(i, end=' ')
    print()
```

Output

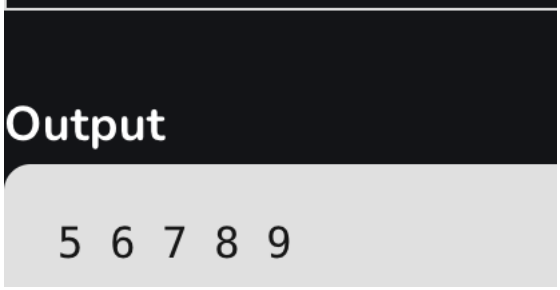
```
1
2 2
3 3 3
4 4 4 4
```

“range()” function:

It is used to generate a sequence of integers within a specified range.

- It is commonly used in loops to control how many times a block of code runs.

```
for n in range(5, 10):
    print(n, end=" ")
```



Loop control statements:

They are special statements that help control the execution of loops (for/while).

- They help modify the default behaviour of the loop, such as stopping early, skipping an iteration, or doing something temporarily.

1. **'*break*' statement**: it is used to exit or "***break***" out of a loop prematurely, before the loop has iterated through all elements or reached its condition. When the "***break***" statement is executed, the program immediately exits the loop and control moves to next line of code after the loop.

```python
# Using For Loop
for i in range(5):
    if i == 3:
        break  # Exit the loop when i is 3
    print(i)

# Using While Loop
i = 0
while i < 5:
    if i == 3:
        break  # Exit the loop when i is 3
    print(i)
    i += 1
```

Output

```
0
1
2
0
1
2
```

2. “***continue*” statement**: it is a loop control statement that forces to execute the next iteration of the loop while skipping the rest of the code inside the loop for the current iteration only; *when the “**continue**” statement is executed in the loop, the code inside the loop following the continue statement will be skipped for the current iteration and the next iteration of the loop will begin.*

```
for i in range(5):
    if i == 3:
        continue  # Skip the rest of the code for i = 3
    print(i)
```

Output

```
0
1
2
4
```

3. **“*pass*” statement**: it is a null operation or a placeholder. It is used when a statement is syntactically required but don’t want to execute any code.

- It does nothing but allows to maintain the structure of the program.

```
for i in range(5):
    if i == 3:
        pass  # Placeholder for future code
    print(i)
```

Output

```
0
1
2
3
4
```

Conditional expressions:

Ternary operators perform conditional checks and assigns values or executes expressions in a single line. It is also known as ‘***conditional expressions***’ because it evaluates a condition and returns o value if condition is True and another value if condition is False.

```python
n = 5
res = "Even" if n % 2 == 0 else "Odd"
print(res)
```

Output

```
Odd
```

```python
age = 20
status = "adult" if age >= 18 else "minor"
print(status)  # adult
```

Match-Case statements:

They are used to perform pattern matching. It allows a value to be compared against multiple patterns and executes the code block associated with the match.

- It provides a more organised alternative to long if-elif-else statements.

```python
def check_number(x):
    match x:
        case 10:
            print("It's 10")
        case 20:
            print("It's 20")
        case _:
            print("It's neither 10 nor 20")

check_number(10)
check_number(30)
```

Output

```
It's 10
It's neither 10 nor 20
```

```python
def num_check(x):
    match x:
        case 10 | 20 | 30:
            print(f"Matched: {x}")
        case _:
            print("No match found")

num_check(10)
num_check(20)
num_check(25)
```

Output

```
Matched: 10
Matched: 20
No match found
```

```python
def num_check(x):
    match x:
        case n if n > 0:
            print("Positive number")
        case n if n < 0:
            print("Negative number")
        case _:
            print("Zero")

num_check(15)
num_check(-8)
num_check(0)
```

Output

```
Positive number
Negative number
Zero
```